

Trabajo Práctico N.º 6

CRUD Nivel 2 — Sistema de Biblioteca (consultas avanzadas y relaciones)

Materia	Bases de Datos Avanzadas
Tema	Operadores lógicos, actualización de arrays, upsert, proyecciones e índices
Modalidad de trabajo	Individual o en parejas
Herramientas	MongoDB Community / MongoDB Atlas, mongosh, MongoDB Compass

1. Introducción

Este trabajo práctico continúa el desarrollo del sistema de biblioteca iniciado en el TP N.º 5. Se incorpora una segunda colección, socios, y una tercera, prestamos, que referencia tanto a socios como a libros, poniendo en práctica el criterio de modelado referenciado estudiado en el TP N.º 1. Sobre este modelo ampliado se profundiza en operadores de consulta lógicos y de arrays, en la actualización condicional (upsert), en la proyección y el ordenamiento de resultados, y en la creación de índices.

Se asume que la colección libros del TP N.º 5 ya existe; si no es así, puede recrearla con un script de carga equivalente al de aquel práctico.

2. Objetivos de aprendizaje

- Modelar una relación referenciada entre tres colecciones (libros, socios y prestamos).
- Combinar condiciones de búsqueda con los operadores lógicos \$and, \$or y \$in.
- Utilizar \$exists y operadores sobre arrays (\$push, \$pull, \$addToSet).
- Aplicar upsert para crear o actualizar un documento en una sola operación.
- Controlar la forma de la salida de una consulta mediante proyección (projection), sort, limit y skip.
- Crear índices simples y compuestos, y verificar su efecto sobre el plan de ejecución.

3. Marco teórico

3.1 Ampliación del modelo: socios y préstamos

Se incorporan dos colecciones nuevas. Un socio de la biblioteca:

```
// Colección "socios"
{
  _id: "S-001",
  nombre: "Martín Pereyra",
  email: "martin.pereyra@correo.com",
  librosFavoritos: ["L-0001", "L-0007"],
  activo: true
}
```

Y un préstamo, que referencia a un socio y a un libro mediante sus respectivos `_id` (modelado referenciado, ya que un préstamo es una entidad con ciclo de vida propio y ambas colecciones referenciadas son compartidas por múltiples préstamos):

```
// Colección "prestamos"
{
  _id: "PR-0001",
  socioId: "S-001",
  libroId: "L-0001",
  fechaPrestamo: "2026-03-01",
  fechaDevolucion: null,
  devuelto: false
}
```

3.2 Operadores lógicos

Operador	Significado	Ejemplo
\$and	Todas las condiciones deben cumplirse (implícito al listar varios campos).	<code>{ \$and: [{ genero: "Novela" }, { anioPublicacion: { \$gt: 1990 } }] }</code>
\$or	Al menos una de las condiciones debe cumplirse.	<code>{ \$or: [{ genero: "Novela" }, { genero: "Ensayo" }] }</code>
\$in	El campo coincide con alguno de los valores de una lista.	<code>{ genero: { \$in: ["Novela", "Poesía"] } }</code>
\$exists	Verifica si un campo está presente (o ausente) en el documento.	<code>{ fechaDevolucion: { \$exists: false } }</code>

3.3 Operadores de actualización sobre arrays

Operador	Efecto
\$push	Agrega un elemento al final de un array.
\$addToSet	Agrega un elemento al array solo si todavía no está presente (evita duplicados).
\$pull	Elimina del array todos los elementos que cumplan una condición.
\$inc	Incrementa (o decrementa, con un valor negativo) un campo numérico.

```
// Agregar un libro favorito sin duplicarlo si ya estaba
db.socios.updateOne(
  { _id: "S-001" },
  { $addToSet: { librosFavoritos: "L-0012" } }
)

// Quitar un libro favorito
db.socios.updateOne(
  { _id: "S-001" },
  { $pull: { librosFavoritos: "L-0007" } }
)
```

3.4 Upsert: crear o actualizar en una sola operación

La opción `{ upsert: true }` indica que, si ningún documento cumple el filtro de un `update`, MongoDB debe crear uno nuevo combinando el filtro con la actualización, en lugar de no hacer nada. Es muy útil para operaciones del tipo "si existe, actualízalo; si no, créalo".

```
db.socios.updateOne(
  { email: "nueva.socia@correo.com" },
  { $set: { nombre: "Lucía Fernández", activo: true }, $setOnInsert: { librosFavoritos: [] } },
  { upsert: true }
)
```

3.5 Proyección, orden y paginación

El segundo argumento de `find()` es la proyección: un documento que indica, con 1 o true, qué campos incluir en el resultado (o, con 0 o false, cuáles excluir). Los métodos `sort()`, `limit()` y `skip()` se encadenan al cursor devuelto por `find()`.

```
db.libros.find(
  { genero: "Novela" },
  { titulo: 1, autor: 1, _id: 0 }
)
.sort({ anioPublicacion: -1 })
.limit(5)
.skip(0)
```

3.6 Índices simples y compuestos

Un índice es una estructura auxiliar que MongoDB mantiene para acelerar las búsquedas por determinados campos, a costa de espacio en disco y de un pequeño costo adicional en cada escritura. Un índice compuesto combina dos o más campos y es especialmente útil cuando las consultas habituales filtran u ordenan por esa combinación.

```
// Índice simple
db.libros.createIndex({ genero: 1 })

// Índice compuesto (género ascendente, año descendente)
db.libros.createIndex({ genero: 1, anioPublicacion: -1 })

// Listar índices existentes
db.libros.getIndexes()
```

4. Consignas de trabajo

Ejercicio 1 — Creación de las colecciones relacionadas

Cree la colección `socios` con al menos 6 documentos y la colección `prestamos` con al menos 10 documentos, referenciando `socios` y `libros` ya existentes mediante sus `_id`. Incluya préstamos con `fechaDevolucion: null` (activos) y otros con fecha de devolución cargada (finalizados).

Ejercicio 2 — Consultas con operadores lógicos

Escriba consultas para: (a) libros del género "Novela" publicados después de 1990, usando `$and` explícito; (b) libros cuyo género sea "Novela" o "Poesía", usando `$or`; (c) el mismo resultado del punto anterior mediante `$in`, y explique por escrito cuándo conviene usar `$or` y cuándo `$in`.

Ejercicio 3 — Préstamos activos con \$exists

Escriba una consulta sobre `prestamos` que devuelva únicamente los préstamos activos (sin fecha de devolución), utilizando `$exists` sobre el campo `fechaDevolucion` o, alternativamente, comparando contra `null`. Documente ambas variantes y verifique que devuelven el mismo resultado.

Ejercicio 4 — Actualización de arrays

Sobre la colección `socios`: (a) agregue un nuevo libro favorito a un socio con `$addToSet`, y repita la misma operación para comprobar que no se duplica; (b) quite un libro favorito con `$pull`; (c) incremente en un campo `cantidadPrestamosHistoricos` (agreguelo si no existe) usando `$inc` cada vez que registre una devolución.

Ejercicio 5 — Upsert

Utilizando `updateOne()` con `upsert: true`, registre un préstamo nuevo identificado por una combinación de `socioId` y `libroId` que todavía no exista en la colección `prestamos`, de forma que si ya existiera se actualizara y si no, se creara. Ejecute el comando dos veces y explique la diferencia de resultado entre la primera y la segunda ejecución.

Ejercicio 6 — Proyección, orden y paginación

Escriba una consulta sobre `libros` que muestre solamente título y autor (ocultando `_id`), ordenada alfabéticamente por título, y que devuelva los resultados de a 3 en 3 simulando una paginación (ejecute la consulta con distintos valores de `skip` para mostrar la "página 1" y la "página 2").

Ejercicio 7 — Índices y su efecto en el rendimiento

Cree un índice compuesto sobre `prestamos` sobre los campos `socioId` y `devuelto`. Antes y después de crearlo, ejecute con `.explain("executionStats")` una consulta que filtre préstamos activos de un socio determinado, y compare la cantidad de documentos examinados (`totalDocsExamined`) y la etapa del plan (`COLLSCAN` vs. `IXSCAN`).

Para pensar

Un índice no siempre mejora el rendimiento: en colecciones muy pequeñas, o para consultas que devuelven casi todos los documentos, el costo de mantenerlo puede no justificarse.

5. Forma de entrega

- Archivo o script con todos los comandos ejecutados, en el orden de los ejercicios.
- Documento (PDF o Word) con las capturas o salidas de consola y las explicaciones solicitadas.
- Plazo de entrega y vía de entrega: a definir por la cátedra en el aula virtual.

6. Criterios de evaluación

Criterio	Descripción	Puntos
Modelo relacionado (referencing)	Las tres colecciones están correctamente relacionadas mediante identificadores coherentes.	15
Operadores lógicos y \$exists	Consultas correctas y bien justificadas, con comprensión de las diferencias entre operadores.	20
Operadores de arrays	Uso correcto de \$push, \$addToSet, \$pull e \$inc.	20
Upsert	Comportamiento de creación vs. actualización correctamente demostrado y explicado.	15
Proyección y paginación	Uso correcto de projection, sort, limit y skip.	15
Índices	Índice compuesto creado correctamente y análisis de explain() coherente con lo observado.	15
Total		100